

MEOP Process

Renovation Status and Toolbox Guide

Current software snapshot, operating model, and migration plan

Purpose. This document summarizes what the renovated MEOP processing toolbox does today, how it is organized, how data are expected to be found, how to install and use it, what is currently tested, and what remains to be ported from Matlab.

Snapshot date	March 22, 2026
Repository state	Python package skeleton plus several scientific workflow ports
Main goal	Make the workflow importable, testable, and progressively independent from Matlab
Current priority already achieved	Matlab is no longer required for package import, runtime data discovery, deployment resolution, or raw ODV import
Primary regression example	Deployment <code>ct88</code> , including the reference file <code>ct88-225-12_lr0_prof.nc</code>
Test suite status	29 tests passed in the current development snapshot

Contents

1	Executive summary	3
2	General description of the toolbox so far	3
2.1	Scientific intent	3
2.2	Current software intent	3
2.3	High-level workflow state	3
3	What has been done	4
3.1	Repository and packaging	4
3.2	Backend separation	4
3.3	Data layout and runtime ownership	5
3.4	Catalog and metadata discovery	5
3.5	Raw ODV import and <code>lr0</code> creation	5
3.6	QC/filter block	5
3.7	High-resolution products and adjustments	5
3.8	Tests and regression harness	6
4	How data are expected to be found	6
4.1	Data classes	6
4.2	Resolution policy	6
4.3	Operational consequence	7
5	Installation	7
5.1	Python environment	7
5.2	Matlab dependency	7
5.3	Configuration	7
5.4	Bootstrap and layout validation	8
6	Usage	8
6.1	Command-line usage	8
6.2	Python API usage	8
6.3	Current processing boundary	8
7	Test cases and validation	9
7.1	Current automated tests	9
7.2	Primary reference case	9
7.3	Validation philosophy	10
8	API overview	10
9	What remains to be done	10
9.1	Deferred scientific processing	10
9.2	Location adjustment	10
9.3	Diagnostics and analyst support	11
9.4	Remaining wrappers that still depend on Matlab	11
9.5	Packaging and deployment polish	11
10	Planned strategy for future developments	11
10.1	Guiding principles	11
10.2	Recommended near-term sequence	11
10.3	Recommended diagnostic roadmap	12

11 Recommended immediate next actions	12
12 Closing note	12

1 Executive summary

The renovation has moved the project from a Matlab-centered operator workspace toward a standard Python package with an explicit runtime layout, a stable public API, a CLI, and a growing set of Python-native scientific steps.

The most important architectural change is that Matlab is no longer required for routine software use cases such as importing the package, locating data, reading configuration, bootstrapping the runtime tree, validating the expected layout, loading deployment metadata, or importing low-resolution raw ODV files. Matlab is now only needed for the workflow stages that have not yet been ported.

At the time of writing, the following workflow elements are already implemented in Python:

- package configuration, runtime path resolution, and data-store bootstrapping;
- deployment and tag catalog loading, including JSON fallback and raw ODV-driven discovery;
- raw ODV import and profile indexing;
- `lr0` generation via a Python replacement for `create_ncargo`;
- the `lr0` QC and filtering block;
- `hr0` generation on a regular 1 dbar grid;
- the `hr1` no-TLC branch;
- delayed-mode `apply_adjustments`;
- comparison and regression helpers used in tests.

The main scientific steps still deferred are the `fr0/fr1` branch, the thermal-lag pathway, the density-stabilization logic associated with that pathway, and most diagnostic and reporting outputs. A placeholder exists for the historical location-adjustment stage so that the workflow boundary remains explicit.

2 General description of the toolbox so far

2.1 Scientific intent

MEOP Process assembles and transforms marine mammal CTD observations into Argo-like profile products and derived outputs used for quality control, adjustment, plotting, and public release. The renovated toolbox preserves that mission while reorganizing the software around a Python package structure.

2.2 Current software intent

The renovated toolbox now has three simultaneous roles:

1. **Operational wrapper** around the legacy Matlab tasks that are still authoritative.
2. **Python-native processing engine** for the workflow parts that have already been ported.
3. **Compatibility layer** that keeps the familiar function names from the historical `python/meop_process.py` entry point while redirecting behavior through the new package.

2.3 High-level workflow state

Workflow element	Current owner	Notes
Configuration loading and path resolution	Python	Repository-local defaults plus optional <code>configs.json</code> machine entries.
Runtime data discovery and validation	Python	No Matlab required.
Sync of deployment/platform JSON metadata	Python	Mirrored into <code>data/config_files/</code> .
Deployment and tag discovery	Python	Uses catalogs, JSON metadata, and raw ODV indexing.
Raw ODV import	Python	Mirrors and extracts from <code>input/<deployment>/</code> .
<code>create_ncargo / lr0</code>	Python	Includes Argo-like NetCDF output generation.
QC/filter block for <code>lr0</code>	Python	Uses Python logic and prefers <code>gsw</code> for density-based criteria.
Location adjustment	Placeholder	Explicit placeholder retained for future work.
<code>hr0</code>	Python	1 dbar interpolation.
<code>hr1</code> no-TLC branch	Python	Available today.
<code>apply_adjustments</code>	Python	Updates existing <code>lr0/lr1/hr0/hr1/fr0/fr1</code> when present.
<code>fr0 / create_fr0_without_lr0</code>	Matlab	Deferred for later port.
TLC pathway and density stabilization	Matlab	Deferred for later port.
<code>hr2</code> generation	Matlab wrapper	Python API exists, implementation still delegated.
Calibration plots, LaTeX report, ODV4 export	Matlab wrapper	Public API exists, implementation still delegated.

3 What has been done

3.1 Repository and packaging

The codebase has been restructured into a standard `src/` layout with a package root at `src/meop_process/`. A `pyproject.toml` file now declares the build system, dependencies, and CLI entry point. A compatibility wrapper remains at `python/meop_process.py` so the historical invocation style is preserved.

The package now exposes a stable public surface through `meop_process.api`. This API keeps the old workflow vocabulary but no longer relies on import-time Matlab startup.

3.2 Backend separation

A backend interface now separates Python-owned workflow stages from Matlab-owned ones:

- `runtime/matlab_backend.py` starts Matlab lazily and only when a Matlab task is actually needed.
- `runtime/python_backend.py` is the execution target for the growing pure-Python path.

This separation is the key enabler for progressive migration.

3.3 Data layout and runtime ownership

The package now makes data resolution explicit. Runtime roots such as `data/tables/`, `data/catalog/`, `data/config_files/`, `data/raw_smru_data_odv/`, and `data/raw_smru_hr_data/` are created and validated directly by Python.

This is especially important because some operational datasets are private and not tracked in GitHub. The package now knows where those datasets are expected to live even when the files themselves are absent from the repository.

3.4 Catalog and metadata discovery

Deployment and tag resolution is now Python-native. The loader can combine information from:

- `list_deployment.csv`;
- `list_deployment_hr.csv`;
- mirrored JSON metadata such as `deployment2.json`, `platform2.json`, `deployment2_patch.json`, and `platform2_patch.json`;
- raw ODV profile indexing when catalog rows are incomplete.

The `ct88` example already demonstrates this behavior: the software can recover deployment and platform information from the JSON metadata and the raw ODV files even when the catalog CSV is incomplete.

3.5 Raw ODV import and `1r0` creation

Low-resolution raw files in ODV form are now discovered, mirrored, parsed, and converted in Python. The main result is a Python implementation of the `create_ncargo` stage that writes `*_1r0_prof.nc` files.

The current port already covers the core Argo-like structure, platform metadata, timestamps, geolocation, parameters, and profile matrices. The `ct88-225-12` regression case is used as the reference target.

3.6 QC/filter block

The first major scientific filter stage has been ported. The current Python implementation reproduces the historical `1r0` QC/filter logic closely enough for the `ct88-225-12` regression case to match the reference file on the key QC arrays and profile counts.

Density-related checks prefer the Python `gsw` package. In environments where `gsw` is not available, a fallback remains in place so that the code is still importable and testable, but the intended direction is to rely on `gsw` as the standard oceanographic engine.

3.7 High-resolution products and adjustments

Two further steps are now Python-native:

- `hr0` generation on the regular 1 dbar grid.
- `hr1` generation for the no-TLC branch.

In addition, `apply_adjustments` is now implemented in Python. It reads runtime coefficient and offset tables, updates adjusted variables and error estimates, and writes those changes back into the available products.

3.8 Tests and regression harness

A real automated test suite now exists. The current snapshot passes 29 tests. These tests cover package configuration, data layout, sync helpers, raw import, ODV parsing, 1r0 generation, QC filtering, hr0, hr1 for the no-TLC branch, adjustments, workflow sequencing, and CLI behavior.

The most valuable regression example is the provided ct88 case, which includes raw ODV input and a reference 1r0 NetCDF file. That example now anchors the scientific validation of the Python ports.

4 How data are expected to be found

A core requirement of the renovation is that the software must know where every class of data belongs, even when those files are not committed to GitHub.

4.1 Data classes

Data class	Canonical location	Examples
Packaged CSV defaults	src/meop_process/resources/tables/ mirrored to data/tables/	table_eff.csv, table_filter.csv, table_meta.csv, table_param.csv
Operator-maintained catalogs	data/catalog/	list_deployment.csv, list_deployment_hr.csv
Mirrored JSON metadata	data/config_files/	deployment2.json, platform2.json, patch files
Incoming raw low-resolution data	input/<deployment>/	ct88_ODV.zip, raw text files
Mirrored/extracted low-resolution working data	data/raw_smru_data_odv/	extracted ODV and CTD/FL text files
High-resolution raw data	data/raw_smru_hr_data/	instrument HR CTD text files resolved from the HR catalog
Reference datasets	references/	WOD, CORA, stable MEOP release, future climatologies
Outputs	final_dataset_prof/ plots/, maps/, doc_latex/ public/<version>/	processed products and reports

4.2 Resolution policy

The package uses explicit resolution rules instead of relying on Matlab working-directory state.

- Tables are looked for first in data/tables/, then in the repository root, then in packaged defaults.
- Catalogs are looked for first in data/catalog/, then in root-level compatibility copies.
- Deployment and platform metadata can be recovered from mirrored JSON files when catalog CSV information is incomplete.
- Raw ODV archives are expected in input/<deployment>/ and are mirrored into data/raw_smru_data_odv/ for actual processing.

4.3 Operational consequence

This means that private or large datasets do not need to be stored inside the GitHub repository. What must be stable is the runtime contract: the package must know where each dataset is expected to appear, and must expose helpers to bootstrap and validate that contract.

5 Installation

5.1 Python environment

A minimal editable installation from the repository root is:

```
python -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install -e .[dev]
```

Current declared core dependencies are:

- numpy
- xarray
- h5netcdf
- scipy
- gsw

5.2 Matlab dependency

Matlab is now optional for several tasks. It is only required when the chosen backend must execute legacy workflow steps that have not yet been ported.

In practical terms:

- **No Matlab needed:** package import, config loading, data bootstrap, layout inspection, deployment discovery, raw ODV import, `lr0`, `hr0`, `hr1` for the no-TLC branch, and `apply_adjustments`.
- **Matlab still needed:** `fr0/fr1`, TLC pathway, `hr2`, calibration plots, LaTeX report generation from the historical Matlab route, and ODV4 export.

5.3 Configuration

The package can operate with repository-local defaults, but it also supports a machine entry from `configs.json`. The loader accepts:

- an explicit process directory;
- an explicit config file;
- a machine key;
- environment variables such as `MEOP_CONFIG_FILE` and `MEOP_MACHINE`.

5.4 Bootstrap and layout validation

Before the first run, the new utilities can prepare and inspect the runtime tree:

```
meop-process --bootstrap-data --show-data-layout
meop-process --validate-data-layout
```

These commands create the expected runtime folders, seed packaged CSV tables, and print or validate the full data layout.

6 Usage

6.1 Command-line usage

The package exposes a console script named `meop-process`. Example calls are:

```
# Import raw ODV data for one deployment
meop-process --deployment ct88 --import_data

# Run the currently available pure-Python path for one tag, no TLC branch
meop-process --smru_name ct88-225-12 --backend python --process_data --notlc

# Apply delayed-mode adjustments to existing products
meop-process --smru_name ct88-225-12 --apply-adjustments
```

6.2 Python API usage

The stable import surface is `meop_process.api`. Example usage:

```
from meop_process.api import (
    bootstrap_data_store,
    describe_runtime_data_layout,
    import_raw_data,
    load_info_deployment,
    process_tags,
)
from meop_process.runtime.python_backend import PythonBackend

bootstrap_data_store()
print(describe_runtime_data_layout(as_text=True))

info = load_info_deployment(deployment="ct88")
print(info["raw_profile_count_by_smru"]["ct88-225-12"])

ok = process_tags(
    deployment="ct88",
    smru_name="ct88-225-12",
    notlc=True,
    backend=PythonBackend(),
)
```

6.3 Current processing boundary

For the main workflow, `process_tags()` currently behaves as follows:

1. prepare runtime directories and sync mirrored metadata;

2. load deployment and tag information in Python;
3. import raw ODV data in Python;
4. clean previous outputs;
5. create `lr0` in Python;
6. run the location-adjustment placeholder;
7. create `hr0` in Python;
8. run Matlab only if required for remaining unported stages;
9. patch metadata and apply delayed-mode adjustments in Python;
10. create `hr1` in Python for the no-TLC branch, or fall back to Matlab for the TLC branch.

7 Test cases and validation

7.1 Current automated tests

The current development snapshot passes 29 tests. The suite covers the following areas:

Area	What is checked
Configuration and packaging	Config loader behavior, CLI wiring, src-layout imports, runtime defaults
Data layout and sync	Runtime data tree bootstrapping, validation, mirrored JSON sync, packaged tables
Catalog resolution	Deployment discovery from CSV and JSON metadata, including fallback cases
Raw ODV import	Import from <code>input/<deployment>/</code> , discovery of CTD/FL/raw files, extraction and indexing
<code>ct88</code> ODV regression	6497 profiles, 19 tags, 306 profiles for <code>ct88-225-12</code> , and preservation of profile structure
<code>lr0</code> regression	Core NetCDF dimensions, variables, metadata, QC arrays, and count attributes against the provided reference file
<code>hr0/hr1</code> regression	Interpolation behavior and no-TLC <code>hr1</code> product creation
Adjustments	Adjusted variables, offset application, error fields, and coefficient metadata
Workflow sequencing	Hybrid Python/Matlab task order and pure-Python no-TLC route

7.2 Primary reference case

The `ct88` deployment is the key reference case used so far. It provides:

- representative raw ODV input;
- deployment and platform metadata examples;
- a reference `ct88-225-12_lr0_prof.nc` output;
- a realistic basis for testing end-to-end parity of the ported workflow stages.

7.3 Validation philosophy

The current strategy is to port a workflow step, then validate it against a reference dataset before moving on. This is preferable to a wholesale rewrite because it keeps scientific equivalence testable at each transition point.

8 API overview

The most important public functions currently exposed through `meop_process.api` are listed below.

Function	Purpose
<code>bootstrap_data_store()</code>	Create runtime folders and seed packaged tables.
<code>describe_runtime_data_layout()</code>	Report the expected data layout, either structured or formatted as text.
<code>validate_runtime_data_layout()</code>	Check required and optional runtime paths.
<code>start_matlab()</code> , <code>stop_matlab()</code>	Explicit backend lifecycle control when Matlab is needed.
<code>load_info_deployment()</code>	Python-native deployment and tag resolution.
<code>update_config_files()</code>	Mirror deployment and platform JSON files into the runtime data area.
<code>import_raw_data()</code>	Mirror and extract raw ODV data from the input tree.
<code>update_metadata()</code>	Patch product metadata from <code>table_meta.csv</code> .
<code>apply_adjustments()</code>	Apply delayed-mode coefficient and salinity-offset adjustments to existing products.
<code>process_tags()</code>	Run the main processing workflow using the chosen backend.
<code>create_hr2()</code>	Public wrapper for the <code>hr2</code> stage; implementation still delegated.
<code>generate_calibration_plots()</code>	Public wrapper for the historical calibration plotting workflow.
<code>generate_doc_latex()</code>	Public wrapper for the historical report-generation workflow.
<code>export_odv4()</code>	Public wrapper for the historical ODV4 export stage.
<code>compare_reference_outputs()</code>	Compare candidate and reference NetCDF outputs.

9 What remains to be done

9.1 Deferred scientific processing

The following core scientific tasks still need work:

- **fr0 and fr1.** These remain on Matlab and are intentionally deferred for now.
- **Thermal-lag correction pathway.** This includes the TLC branch itself and the related `apply_tlc_fr` logic.
- **Density-stabilization logic.** The complex Matlab function used in the TLC/high-resolution pathway has not been ported yet.

9.2 Location adjustment

The historical location-adjustment step is currently represented as a placeholder. This is deliberate: the workflow boundary is preserved, but the implementation is postponed until the surrounding priorities are ready.

9.3 Diagnostics and analyst support

A major remaining area is the diagnostic layer used to inspect the data and derive adjustments from historical profiles and climatological references. This work has barely started and will likely require dedicated design.

The likely ingredients are:

- Python-native profile diagnostics and section views;
- historical reference-data comparison tools;
- climatology-based diagnostics;
- analyst notebooks or structured reports;
- eventually a reproducible workflow to propose and review delayed-mode adjustments.

9.4 Remaining wrappers that still depend on Matlab

The public functions already exist for `create_hr2`, calibration plots, LaTeX report generation, and ODV4 export, but the underlying implementations still delegate to Matlab.

9.5 Packaging and deployment polish

A few software-level tasks also remain:

- broaden dependency declarations once diagnostics and mapping mature;
- add richer documentation around reference datasets and private-data mounting conventions;
- create release notes and versioning rules for future public package releases;
- decide how much backward compatibility to preserve in filenames, metadata, and CLI behavior.

10 Planned strategy for future developments

10.1 Guiding principles

The next stages should follow these principles:

1. Preserve scientific behavior before optimizing implementation details.
2. Keep the public API stable while internals are migrated.
3. Prefer small, testable ports over large rewrites.
4. Use reference cases and comparison harnesses for every scientific transition.
5. Keep data-location rules explicit and package-owned.

10.2 Recommended near-term sequence

Phase	Recommended scope
Phase 1	Consolidate the current Python core, refresh internal documentation, and test with Matlab as soon as a license is available. Confirm parity of <code>1r0</code> , <code>QC</code> , <code>hr0</code> , <code>hr1</code> no-TLC, and <code>apply_adjustments</code> .

Phase	Recommended scope
Phase 2	Design the diagnostic layer: data-inspection plots, profile comparisons, and climatology interfaces. This will likely shape the future analyst workflow more than any remaining low-level refactor.
Phase 3	Port <code>fr0/fr1</code> only when the surrounding data model and diagnostics are already stable.
Phase 4	Revisit the TLC and density-stabilization pathway, likely as its own dedicated effort because scientific parity matters more than mechanical translation.
Phase 5	Port or redesign <code>hr2</code> , report generation, calibration plots, and ODV export once the underlying processing chain is settled.

10.3 Recommended diagnostic roadmap

Because diagnostics will be central for future delayed-mode work, a likely roadmap is:

1. create reusable profile-selection and plotting utilities in Python;
2. build interfaces to historical reference datasets and climatologies;
3. define a standard comparison bundle for one tag: track map, T/S view, profile overlays, offset estimates, and candidate coefficients;
4. add regression tests for diagnostic input preparation and summary statistics;
5. only then decide whether the final analyst interface should be a CLI report, notebook workflow, or a small application.

11 Recommended immediate next actions

Given the current state, the most useful immediate actions are:

- test the current snapshot against Matlab as soon as the license is available;
- verify the Python outputs on a few deployments beyond `ct88`;
- document the expected private-data mounting conventions for catalogs, raw data, and references;
- start designing the diagnostic layer before reopening the deferred `fr0` and TLC tasks.

12 Closing note

The renovation has already crossed the most important software threshold: MEOP Process is no longer locked inside Matlab just to be imported, configured, or pointed at data. The scientific core is not fully migrated yet, but the project now has a clean package skeleton, explicit data contracts, a test suite, and a validated reference case. That creates a much safer foundation for the next development stages.